Projet :
Optimisation d'un problème Mixte non-contraint Continue-Binaire quadratique (mixQUBO)

Master Informatique 1 - année 2025-2026

1er avril 2026

1 Consignes

Le code développé est à rendre par le git de l'université. Un rapport (au format pdf) est attendu avec la description de vos algorithmiques, la méthodologie expérimentale et l'exposé des résultats accompagnés de commentaires.

2 Contexte

Les problèmes quadratiques non-contraint à variables binaires (Quadratic Unconstrained Binary Problem - QUBO) sont des problèmes combinatoires génériques NP-difficiles qui modélisent des nombreuses applications [1]. Ces problèmes sont aussi les modèles d'entrée de nombreux algorithmes exécutés sur les nouveaux ordinateurs quantiques. Ainsi, de nombreuses recherches tentent de trouver la meilleure modélisation sous forme QUBO, ou tentent de développer des algorithmes d'optimisation sur machines classiques, sur machines quantiques ou de manière hybride (quantique/classique).

Par ailleurs, la résolution des problèmes d'optimisation à variable mixte, c'est-à-dire ayant des variables continues (nombres réels) et des variables discrètes (par exemple à valeur booléenne), reste des problèmes ouverts avec de nombreuses applications dans l'industrie [2]. Les stratégies d'optimisation doivent trouver un compromis d'exploration/exploitation des différents types de variables continues et discrètes de manière indépendante, successive ou conjointe.

À ce jour, il n'existe pas problème quadratique non-contraint à variables mixtes. Nous proposons dans ce problème une possibilité d'un tel type de problème que nous appelerons **mixQUBO** pour Mixed Quadratic Unscontrainted Continous and Binary Problem Optimization.

Le but du projet est de développer des algorithmes d'optimisation performants sur les problèmes mixQUBO en utilisant des techniques d'optimisation combinatoire et numérique.

2.1 Définition du problème QUBO à variable binaire

Une instance du problème QUBO[1] [1] de dimension n est définie à partir d'une (demi) matrice de nombres entiers (positifs ou négatifs) $J = (J_{ij})_{i,j\in\{1,\dots,n\}}$. Seuls les nombres J_{ij} tels que $i < j$ sont non nuls. La fonction objective f à **minimiser** est définie de la manière suivante lorsque les variables binaires x_i appartiennent à $\{0, 1\}$:

$$\forall x \in \{0,1\}^n,\ f_{\text{QUBO}}(x) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} J_{ij}(-1)^{x_i}(-1)^{x_j}$$

Ainsi si $x_i = 0$ alors le terme $(-1)^{x_i} = 1$, et si $x_i = 1$ alors le terme $(-1)^{x_i} = -1$:

Bool.	x_i	$(-1)^{x_i}$
False	0	1
True	1	-1

[1] Dans ce projet, le problème QUBO est purement quadratique sans terme linéaire

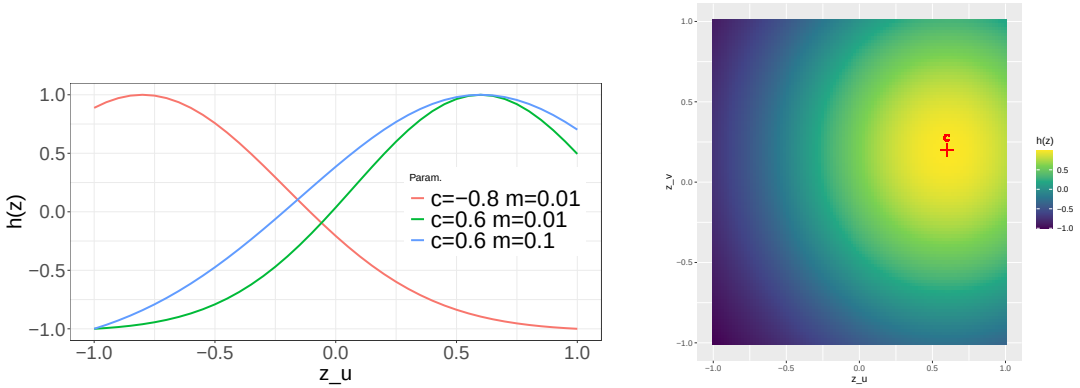


Figure 1: À gauche, exemples de fonctions $h_{c,m}(z_u, z_v)$ pour différentes valeur de c_1 et m lorsque $z_v = c_2$. À droite, représentation 3d de la fonction $h(z_u, z_v)$ pour $c = (0.6, 0.2)$ et $m = 0.1$. Le maximum est en $z_u = c_1$ et est de valeur 1, le minimum est en $z_u = -1$ et est de valeur -1.

2.2 Définition du problème mixQUBO à variable mixte

Nous vous proposons une version à variable mixte du problème QUBO que nous appellerons mixQUBO. Le principe est de faire dépendre l'interaction $J_{i,j}$ entre les variables x_i et x_j, de 2 variables continues z_u et z_v: $J_{ij} = J_{u,v,i,j}\ h(z_u, z_v)$ avec h une fonction telle que $h(z_u, z_v) \in [-1, 1]$ et $J_{u,v,i,j}$ un nombre entier positif ou négatif.

Ainsi, le problème mixQUBO dépend de n variables binaires avec chaque variable binaire telle que $x_i \in \{0, 1\}$, et d variables continues avec chaque variable continues telle que $z_u \in [-1, 1]$. Comme pour le problème QUBO, le problème mixQUBO est un problème de **minimisation** qui est la somme de p termes. Chaque terme dépend de la valeur de 2 variables binaires i_k et j_k, et de 2 variables continues u_k et v_k. Chaque terme est paramétré par un entier J_k, un vecteur de dimension 2 $c_k \in [-1, 1]^2$ et d'un nombre réel $m \in]0, 1[$. La définition complète est la suivante :

$$\forall z \in [-1,1]^d \text{ et } \forall x \in \{0,1\}^n,\ f(x) = \sum_{k=1}^{p} J_k\ h_{c_k,m_k}(z_{u_k}, z_{v_k})\ (-1)^{x_{i_k}} (-1)^{x_{j_k}}$$

avec $h_{c_k,m_k} : [-1, 1]^2 \to [-1, 1]$ une fonction à deux variables à valeur dans $[-1, 1]$ définie par :

$$\forall z_u \in [-1,1] \text{ et } \forall z_v \in [-1,1],\ h_{c,m}(z_u, z_v) = \frac{2e^{-\alpha((z_u-c_1)^2+(z_v-c_2)^2)}}{1-m} - \frac{1+m}{1-m}$$

avec $\alpha = \frac{-\log m}{(\pm 1-c_1)^2+(\pm 1-c_2)^2}$.

Le paramètre c est la position du maximum (égale à 1) et le paramètre m régle la concavité de la courbe. Le minimum de la fonction h est -1 à l'une des extrémités du domaine (voir graphique 1). À noter que la fonction h a la forme d'une densité Gaussienne dont la modalité est centrée en c et la variance est paramétrée par la valeur m.

2.3 Interprétation

Une interprétation possible du problème QUBO est d'imaginer que le but est de minimiser l'énergie totale d'un système "magnétique" où les variables binaires représentent le spin magnétique (Nord/Sud) de chaque élément du système. Un spin d'un élément peut être soit positif égale à $+1$, soit négatif égale à -1. Par convention, lorsque $x_i = 0$ le spin est $+1$, et lorsque $x_i = 1$ le spin est -1. L'énergie totale du systèmes est la somme des énergies d'interaction entre chaque paire d'éléments. L'énergie d'interaction entre les éléments i et j est définie par $J_{ij}(-1)^{x_i}(-1)^{x_j}$. L'énergie J_{ij} est ajoutée lorsque les spins sont de même signe.

La figure 2 donne un exemple pour la dimension $n = 4$ d'un problème QUBO. La matrice Q correspondante sont égale à :

$$J = \begin{bmatrix} 0 & -1 & 1 & -1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

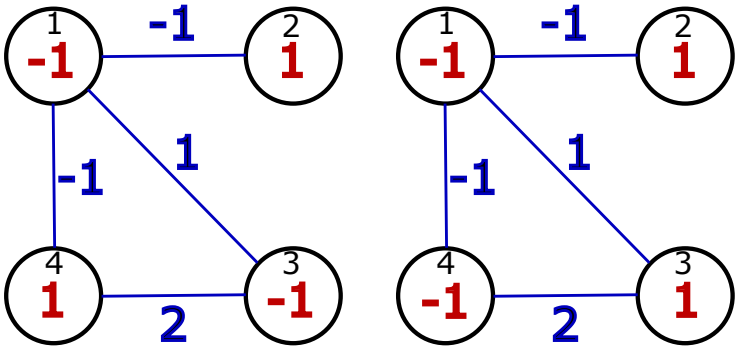


Figure 2: Exemple d'instance de problème QUBO de dimension $n = 4$. Les identifiants des éléments sont écrits en noir à l'intérieur des nœuds, les valeurs des énergies J_{ij} sont représentées en bleu sur les arêtes et la valeur des spins des éléments en rouge. À gauche, la solution $x = 1010$ a pour fitness $f(x) = 1$. À droite le minimum global du problème avec la solution $x^\star = 1001$ ayant pour fitness $f(x^\star) = -3$.

Table 1: Paramètres des instances `mxqubo_v0_id.json`.

id	d	n	n_cont	n_bin
0	10	100	10	250
1	10	100	20	250
2	10	100	10	500
3	10	100	20	500
4	20	100	20	250
5	20	100	40	250
6	20	100	20	500
7	20	100	40	500
8	10	200	10	500
9	10	200	20	500
10	10	200	10	1000
11	10	200	20	1000
12	20	200	20	500
13	20	200	40	500
14	20	200	20	1000
15	20	200	40	1000

Les interactions entre les éléments 2 et 3 ($J_{23} = 0$) et les éléments 2 et 4 ($J_{24} = 0$) sont nulles. La solution représentée à gauche est la solution $x_1 = 1010$ ayant respectivement pour spins $-1, 1, -1, 1$. La fitness de cette solution est donc : $f(x) = -J_{12} + J_{13} - J_{14} - J_{34} = 1 + 1 + 1 - 2 = 1$. La solution représentée à droite est la solution $x_2 = 1001$ ayant respectivement pour spins $-1, 1, 1, -1$. La fitness de cette solution est donc : $f(x^\star) = -J_{12} - J_{13} + J_{14} - J_{34} = 1 - 1 - 1 - 2 = -3$.

Le problème mixQUBO ajoute sur chaque énergie d'interaction une dépendance avec un "champ extérieur" qui dépend de variables continues. La fonction $h_{c,m}$ dont la valeur est comprise entre -1 et 1 peut être interprétée comme un facteur qui réduit ou inverse l'énergie d'interaction J_{ij}. Ce facteur dépend de $d^2 = (z_u - c_1)^2 + (z_v - c_2)^2$, c'est-à-dire le carré de la distance entre le point $c \in \mathbb{R}^2$ avec le point de coordonnées (z_u, z_v).

3 Méthodologie

Les algorithmes seront testés et comparés sur les 16 instances `mxqubo_v0_id.json` (avec `id` de 0 à 15) contenues dans le répertoire `instances`. Les instances sont générées avec un nombre de termes égale à n_bin $\times$ n_cont où n_bin est le nombre de couples de variable binaire $(i, j) \in \{0, n-1\}^2$ sélectionnés (sans remise), et n_cont est le nombre termes par couple (i, j) sélectionné. Les variables (u, v), ainsi que les paramètres c et m sont obtenus aléatoirement. Les paramètres de ces instances sont définis avec deux dimensions de l'espace continue $d \in \{10, 20\}$, deux dimensions de l'espace binaire $n \in \{100, 200\}$, deux valeurs de n_bin $\in \{2.5n, 5n\}$ et deux valeurs de n_cont $\in \{d, 2d\}$. Les paramètres des 16 instances sont résumés dans le tableau 1.

Les algorithmes devront tous avoir le même temps de calcul, *i.e.* 15 secondes de calcul sur la machine "étalon". Pour connaître l'équivalent du temps sur votre machine de la machine étalon, compiler le code fournis (voir gitlab : `https://gitlab.dpt-info.univ-littoral.fr/m1-wedsci/projet_m1_optimisation_2026`) et exécuter le. Le temps d'exécution sur la machine étalon est de 10 secondes. Vous pouvez alors calculer le ratio du temps d'exécution sur votre propre machine. Indiquer dans votre rapport, le temps d'exécution du code étalon sur votre machine. Les algorithmes doivent s'exécuter sur un temps égale à 15 secondes sur la machine étalon.

Les algorithmes qui par nature sont stochastiques, devront être tester au moins sur 30 exécutions.

4 Techniques d'optimisation combinatoire

Dans cette section, nous vous proposons de mettre au point des algorithmes d'optimisation combinatoire pour résoudre la partie QUBO du problème mixQUBO. Dans cette section, le vecteur de variables continues sera toujours égale au vecteur nul de dimension d: $z = [0.0, 0.0, \ldots, 0.0]$.

Questions :

2.1 Initialisation :

2.1.a Coder une recherche aléatoire.

2.2.b Tester et commenter les résultats de votre recherche aléatoire.

2.3 Recherches locales :

2.3.a Coder au moins 2 métaheuristiques.

2.3.c Comparer les performances de vos algorithmes.

5 Techniques d'optimisation continue

De manière symétrique à la section précédente, dans cette section, nous vous proposons de mettre au point des algorithmes d'optimisation continue pour résoudre la partie continue du problème mixQUBO. Dans cette section, le vecteur de variables binaires sera toujours égale au vecteur nul de dimension n: $x = 000\ldots00$.

Questions :

2.1 Initialisation :

2.1.a Coder une recherche aléatoire.

2.2.b Tester et commenter les résultats de votre recherche aléatoire.

2.3 Recherches locales :

2.3.a Coder au moins 2 méthodes d'optimisation.

2.3.c Comparer les performances de vos algorithmes.

6 Techniques d'optimisation mixte

Dans cette section, nous vous proposons de proposer et de mettre au point des algorithmes pour résoudre le problème mixQUBO. Dans cette section, les variables continues et binaires sont à optimiser.

Questions :

2.1 Initialisation :

2.1.a Coder une recherche aléatoire.

2.2.b Tester et commenter les résultats de votre recherche aléatoire.

2.3 Recherches locales :

2.3.a Proposer au moins deux méthodes d'optimisation pour le problème mixQUBO

2.3.c Comparer les performances de vos algorithmes.

References

[1] Gary Kochenberger, Jin-Kao Hao, Fred Glover, Mark Lewis, Zhipeng Lü, Haibo Wang, and Yang Wang. The unconstrained binary quadratic programming problem: a survey. *Journal of combinatorial optimization*, 28:58–81, 2014.

[2] Thi Thoi Tran. *Optimisation Non Linéaire en Variables Mixtes Continues Et Discrètes Pour Des Simulateurs de Type de Boîte Noire*. PhD thesis, 2021.